

Exercise: GPU/SIMT Programming

CUDA Indexing, SIMT Divergence, and Bandwidth

Assume a CUDA GPU with warp size 32, and assume `sizeof(float) == 4`. Consider the following CUDA kernel:

```
__global__ void vecTransform(float *x, float *y, float *out, size_t n) {
    size_t idx = blockDim.x * blockIdx.x + threadIdx.x;

    if (idx < n) {
        float v = x[idx] + y[idx];

        if (idx % 2 == 0)
            out[idx] = 2.0f * v;
        else
            out[idx] = v + 1.0f;
    }
}
```

The host launches:

```
vecTransform<<<16, 64>>>(d_x, d_y, d_out, 1000);
```

Answer all subproblems. Show your reasoning where appropriate.

1. How many CUDA threads are launched in total? How many of them actually write to `out`?
2. For `blockIdx.x = 5` and `threadIdx.x = 17`, what value of `idx` does the thread compute?
3. Which threads in the last block do no useful work because of the `idx < n` guard?
4. Consider the first warp of block 0, whose `idx` values are 0 through 31. Does the inner `if (idx % 2 == 0)` cause SIMT thread divergence? Explain how many lanes are active on each branch.
5. Consider the second warp of the last block, whose `idx` values are 992 through 1023. Explain the combined effect of the outer bounds check and the inner even/odd branch.
6. Rewrite the CUDA kernel as an equivalent traditional single-threaded C++ function that runs on the CPU. Your function should have the following signature:

```
void vecTransform_cpu(const float *x, const float *y, float *out, size_t n);
```

7. For each valid element, assume the kernel reads `x[idx]`, reads `y[idx]`, writes `out[idx]`, and performs 2 floating-point operations. Ignoring cache effects, estimate the operational intensity in FLOPs per byte. Is this kernel more likely to be compute-bound or bandwidth-bound? Briefly justify.